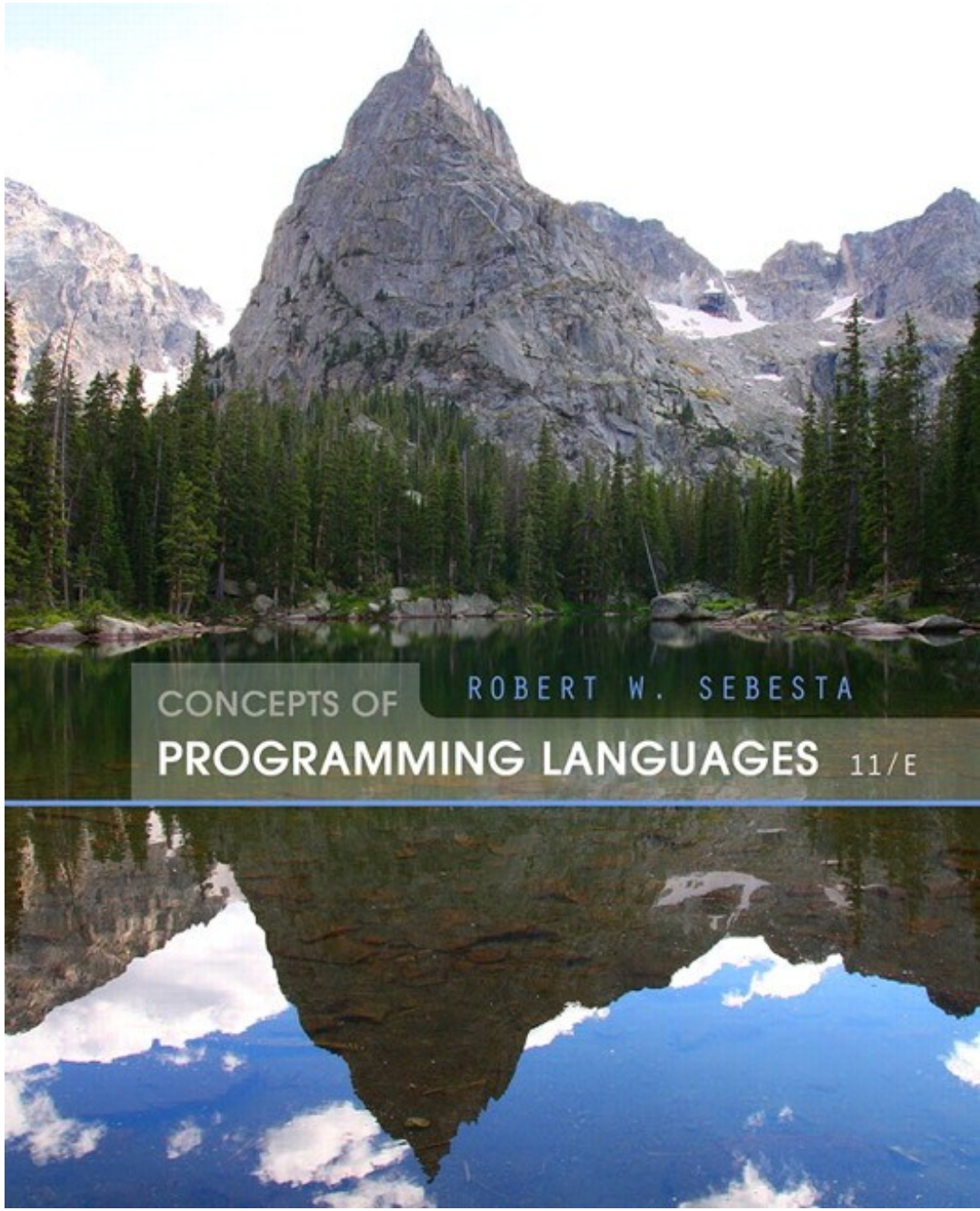
Bölüm 6

Veri Türleri (Data Types)



ISBN 0-321-49362-1

Bölüm 6 Konular

- 6.1 Giriş
- 6.2 İlkel Veri Türleri
- 6.3 Karakter Dizisi Türleri (Character String Types)
- 6.4 Sayım Türleri (Enumeration Types)
- 6.5 Dizi Türleri (Array Types)
- 6.6 İlişkisel Diziler (Associative Arrays)
- 6.7 Kayıt Türleri (Record Types)
- 6.8 Demet Türleri (Tuple Types)
- 6.9 Liste Türleri (List Types)
- 6.10 Birlik Türleri (Union Types)
- 6.11 İşaretçi ve Referans Türleri (Pointer and Reference Types)
- 6.12 Tür Kontrolü (Type Checking)
- 6.13 Güçlü Tipleme (Strong Typing)
- 6.14 Tip Eşdeğerliği (Type Equivalence)
- 6.15 Teori ve Veri Türleri (Theory and Data Types)

Copyright © 2015 Pearson. All rights reserved.

6.1 Giriş

- Bir *veri türü (data type)*, veri nesnelerinin bir koleksiyonunu ve bu nesneler üzerinde önceden tanımlanmış bir dizi işlemi tanımlar.
- *Tanımlayıcı (descriptor)*, bir değişkenin özniteliklerinin toplamıdır.
- Bir *nesne (object)*, kullanıcı tanımlı (soyut veri, abstract data) türünün bir örneğini temsil eder.
- Tüm veri türleri için tek bir tasarım sorunu: Hangi işlemler tanımlanmıştır ve bunlar nasıl belirtilmiştir?

Copyright © 2015 Pearson. All rights reserved.

6.2 İlkel Veri Türleri

- Hemen hemen tüm programlama dilleri bir dizi *ilkel veri türü (primitive data types)* sağlar.
- İlkel veri türleri: Diğer veri türleri açısından tanımlanmamış olanlardır.
- Bazı ilkel veri türleri yalnızca donanımın yansımasıdır.
- Diğerleri ise uygulamaları (implementation) için sadece az miktarda donanım dışı destek gerektirir.

Copyright © 2015 Pearson. All rights reserved.

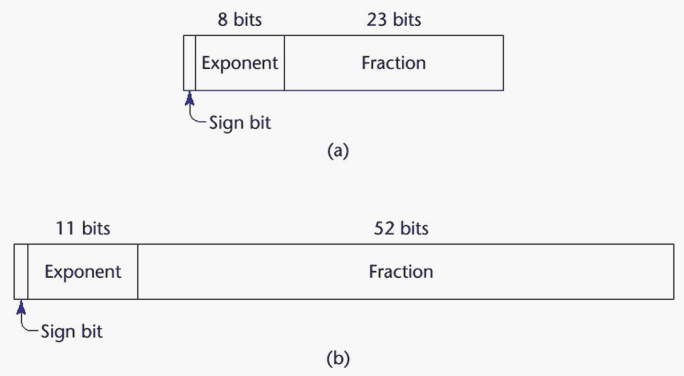
6.2.1 İlkel Veri Türleri: Integer

- Neredeyse her zaman donanımın tam bir yansımasıdır, bu nedenle haritalandırma önemsizdir.
- Bir dilde sekiz farklı tamsayı türü olabilir.
- Java'nın işaretli tamsayı boyutları: **`byte`**, **`short`**, **`int`**, **`long`**

Copyright © 2015 Pearson. All rights reserved.

6.2.2 İlkel Veri Türleri: Floating Point

- Gerçek sayıları modeller, ancak yalnızca yaklaşımlar olarak.
- Bilimsel kullanım için diller en az iki kayan nokta türü destekler (örneğin, **`float`** ve **`double`**; bazen daha fazlası).
- Genellikle donanımla tamamen aynıdır, ancak her zaman böyle değildir.
- IEEE Floating-Point Standard 754



Copyright © 2015 Pearson. All rights reserved.

6.2.3 İlkel Veri Türleri: Complex

- Bazı diller complex type destekler, örneğin C99, Fortran ve Python.
- Her değer, reel kısım ve imajiner kısım olmak üzere iki float değerinden oluşur.
- Literal form (Python'da):

  `(7 + 3j)`, burada `7` reel kısım, `3` ise imajiner kısımdır.

Copyright © 2015 Pearson. All rights reserved.

6.2.4 İlkel Veri Türleri: Decimal

- İş uygulamaları için (para)
  - COBOL için gerekli
  - C# decimal veri türü sunar
- Sabit sayıda ondalık basamakları kodlanmış biçimde (BCD) saklayın.
- *Avantaj*: doğruluk
- *Dezavantaj*: sınırlı alan, bellek israfı

Copyright © 2015 Pearson. All rights reserved.

6.2.5 İlkel Veri Türleri: Boolean

- En basit olanı
- Değer aralığı: iki unsur, biri “ true ” (doğru) ve diğeri “ false ” (yanlış) için
- Bit olarak uygulanabilir, ancak genellikle byte olarak uygulanır.
  - Avantaj: okunabilirlik

Copyright © 2015 Pearson. All rights reserved.

6.2.6 İlkel Veri Türleri: Character

- Sayısal kodlar olarak saklanır
- En yaygın kullanılan kodlama: ASCII
- Alternatif, 16 bit kodlama: Unicode (UCS-2)
  - Çoğu doğal dildeki karakterleri (characters) içerir
  - Başlangıçta Java'da kullanılıyordu
  - C# ve JavaScript de Unicode'u destekler.
- 32-bit Unicode (UCS-4)
  - Fortran tarafından desteklenir, 2003'ten itibaren

Copyright © 2015 Pearson. All rights reserved.

6.3 Character String Types

- Değerler karakter dizileridir.
- Tasarım sorunları:
  - Bu bir ilkel tür mü yoksa sadece özel bir dizi türü mü?
  - Dizilerin uzunluğu statik mi yoksa dinamik mi olmalı?

Copyright © 2015 Pearson. All rights reserved.

6.3.1 Character String Types Operations

- Tipik operasyonlar:
  - Atama ve kopyalama
  - Karşılaştırma (=, > vb.)
  - Zincirleme (Catenation)
  - Substring referansı
  - Desen eşleştirme

Copyright © 2015 Pearson. All rights reserved.

6.3.2 Belirli Dillerde Character String Type

- C ve C++
  - İlkel değil
  - **char** dizileri ve işlemler sağlayan bir fonksiyon kütüphanesi kullanır
- SNOBOL4 (bir string manipülasyon dili)
  - İlkel
  - Karmaşık desen eşleştirme dahil birçok işlem
- Fortran ve Python
  - Atama ve çeşitli işlemler içeren ilkel tip
- Java
  - `String` sınıfı aracılığıyla ilkel
- Perl, JavaScript, Ruby, ve PHP
  - Regular expressions kullanarak yerleşik desen eşleştirme sağlar

Copyright © 2015 Pearson. All rights reserved.

6.3.4 Character String Uzunluk Seçenekleri

- Statik: COBOL, Java'nın `String` sınıfı
- *Sınırlı Dinamik Uzunluk*: C ve C++
  - Bu dillerde, stringin uzunluğunu korumak yerine, stringin karakterlerinin sonunu belirtmek için özel bir karakter kullanılır.
- *Dinamik* (maksimum yok): SNOBOL4, Perl, JavaScript

Copyright © 2015 Pearson. All rights reserved.

6.3.5 Character String Type Değerlendirmesi

- Yazılabilirliğe yardım
- Statik uzunluklu bir ilkel tür olarak, bunları sağlamak ucuzdur - neden olmasın?
- Dinamik uzunluk güzel, ama masrafına değer mi?

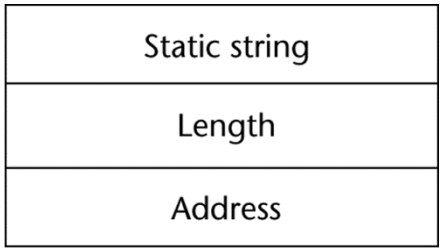
Copyright © 2015 Pearson. All rights reserved.

6.3.6 Character String Implementation

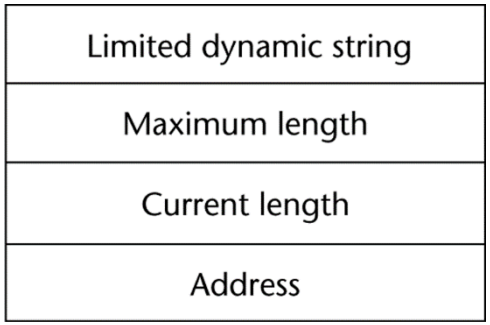
- Statik uzunluk: derleme zamanı tanımlayıcısı
- Sınırlı dinamik uzunluk: uzunluk için çalışma zamanı tanımlayıcısı gerekebilir (ancak C ve C++'da gerekmez)
- Dinamik uzunluk: çalışma zamanı tanımlayıcısı gerekir; allocation/deallocation en büyük implementation sorunudur

Copyright © 2015 Pearson. All rights reserved.

6.3.7 Compile ve Run-Time Descriptors



Statik stringler için derleme zamanı tanımlayıcısı



Sınırlı dinamik stringler için çalışma zamanı tanımlayıcı

Copyright © 2015 Pearson. All rights reserved.

6.3.8 Kullanıcı Tanımlı Sıralı (Ordinal) Türler

- *Sıralı tür (ordinal type)*, olası değerlerin aralığının pozitif tamsayılar kümesiyle kolayca ilişkilendirilebildiği türdür.
- Java'da ilkel sıralı türlere örnekler
  - **`integer`**
  - **`char`**
  - **`boolean`**

Copyright © 2015 Pearson. All rights reserved.

6.4 Enumeration Types

- Sabitler (constants) olarak adlandırılan tüm olası değerler tanımda verilmiştir.
- C# örneği

```
enum days {mon, tue, wed, thu, fri, sat, sun};
```

- Tasarım sorunları
  - Bir enumeration sabiti birden fazla tip tanımında görünebilir mi? Eğer öyleyse, bu sabitin bir örneğinin tipi nasıl kontrol edilir?
  - Enumeration değerleri tamsayıya zorlanıyor mu?
  - Diğer herhangi bir tür, bir enumeration türüne zorlanmış mı?

Copyright © 2015 Pearson. All rights reserved.

6.4.1 Enumerated Type Değerlendirilmesi

- Okunabilirliği artırma, örneğin bir rengi sayı olarak kodlamaya gerek olmaması
- Güvenilirliğe yardımcı olur, örneğin derleyici şunları kontrol edebilir:
  - operasyonlar (renklerin eklenmesine izin vermez)
  - Hiçbir enumeration değişkenine tanımlanmış aralığı dışındaki bir değer atanamaz.
  - C# ve Java 5.0, C++'dan daha iyi bir enumeration desteği sunar çünkü bu dillerde enumeration tipi değişkenler tamsayı tiplerine zorlanmaz.

Copyright © 2015 Pearson. All rights reserved.

6.5 Array Types

- Array, tek tek elemanların, ilk elemana göre topluluk içindeki konumlarıyla tanımlandığı homojen bir veri elemanları topluluğudur.

Copyright © 2015 Pearson. All rights reserved.

6.5.1 Array Tasarım Sorunları

- Alt simgeler (subscript) için hangi türler yasaldır?
- Öğe referanslarındaki alt simge ifadeleri aralık kontrolünden geçirilir mi?
- Alt simge aralıkları ne zaman sınırlıdır?
- Allocation ne zaman gerçekleşir?
- Düzensiz veya dikdörtgen çok boyutlu dizilere izin verilir mi, yoksa her ikisine de izin verilir mi?
- En fazla kaç tane alt simge olabilir?
- Array nesneleri başlatılabilir (initialize) mi?
- Her türlü dilim destekleniyor mu?

Copyright © 2015 Pearson. All rights reserved.

6.5.2 Array Indexing

- *İndeksleme* (veya alt simge ekleme, subscripting), indekslerden elemanlara yapılan bir eşlemedir.

  array_name (index_value_list) → an element
- Index Syntax
  - Fortran ve Ada parantez kullanır.
    - Ada, array referansları ve fonksiyon çağrıları arasında tekdüzelik olduğunu göstermek için açıkça parantez kullanır, çünkü her ikisi de eşlemelerdir.
  - Diğer dillerin çoğu köşeli parantez kullanır.

Copyright © 2015 Pearson. All rights reserved.

6.5.3 Arrays Index (Subscript) Types

- FORTRAN, C: sadece integer
- Java: sadece integer türleri
- İndeks aralığı kontrolü
  - C, C++, Perl ve Fortran aralık kontrolünü belirtmez.
  - Java, ML, C# aralık kontrolünü belirtir.

Copyright © 2015 Pearson. All rights reserved.

6.5.4 Subscript Binding ve Array Kategorileri

- *Static*: subscript aralıkları statik olarak bağlanır ve storage allocation statiktir (run-time öncesinde)
  - Avantaj: verimlilik (dinamik allocation yok)
- *Fixed stack-dynamic*: subscript aralıkları statik olarak bağlanır, ancak allocation declaration sırasında yapılır
  - Avantaj: alan verimliliği

Copyright © 2015 Pearson. All rights reserved.

6.5.4 Subscript Binding ve Array Kategorileri (Devam Ediyor)

- *Fixed heap-dynamic*: fixed stack-dynamic'e benzer: storage binding dinamiktir ancak allocation edildikten sonra sabitlenir (yani, binding talep edildiğinde yapılır ve depolama stack'ten değil, heap'ten allocate edilir)

Copyright © 2015 Pearson. All rights reserved.

6.5.4 Subscript Binding ve Array Kategorileri (Devam Ediyor)

- Heap-dynamic: subscript aralıklarının binding edilmesi ve depolama allocation'ı dinamiktir ve istediğiniz kadar değiştirilebilir
  - Avantaj: esneklik ("Arrays" program yürütme sırasında büyüyebilir veya küçülebilir)

Copyright © 2015 Pearson. All rights reserved.

6.5.4 Subscript Binding ve Array Kategorileri (Devam Ediyor)

- `static` değiştirici içeren C ve C++ dizileri statiktir.
- `static` değiştirici içermeyen C ve C++ dizileri fixed stack-dynamic'tir.
- C ve C++, fixed heap-dynamic arrays sağlar.
- C#, fixed heap-dynamic sağlayan ikinci bir dizi sınıfı olan `ArrayList` içerir.
- Perl, JavaScript, Python ve Ruby, heap-dynamic arrays destekler.

Copyright © 2015 Pearson. All rights reserved.

6.5.5 Array Initialization

- Bazı diller, depolama alanı allocation'ı sırasında initialization'a izin verir.
  - C, C++, Java, C# örneği

```
int list [] = {4, 5, 7, 83}
```

  - C ve C++ character strings

```
char name [] = "freddie";
```

  - C ve C++ strings array'i

```
char *names [] = {"Bob", "Jake", "Joe"];
```

  - String nesnelerinin Java ile initializate edilmesi

```
String[] names = {"Bob", "Jake", "Joe"};
```

Copyright © 2015 Pearson. All rights reserved.

6.5.5 Array Initialization

- C tabanlı diller
  - `int list [] = {1, 3, 5, 7}`
  - `char *names [] = {"Mike", "Fred", "Mary Lou"};`
- Python
  - List comprehensions

```
list = [x ** 2 for x in range(12) if x % 3 == 0]
   puts [0, 9, 36, 81] in list
```

Copyright © 2015 Pearson. All rights reserved.

6.5.6 Heterojen Arrays

- *Heterojen dizi (heterogeneous array)*, öğelerinin aynı türde olması gerekmeyen dizidir.
- Perl, Python, JavaScript ve Ruby tarafından desteklenir.

Copyright © 2015 Pearson. All rights reserved.

6.5.7 Arrays Operations

- APL, vektörler ve matrisler için en güçlü array işleme işlemlerinin yanı sıra tekli operatörler (örneğin, sütun öğelerini tersine çevirmek için) sağlar.
- Python'un array atamaları, ancak bunlar yalnızca referans değişiklikleridir. Python ayrıca array birleştirme (catenation) ve eleman üyelik işlemlerini de destekler.
- Ruby ayrıca array birleştirme (catenation) özelliği de sunar.

Copyright © 2015 Pearson. All rights reserved.

6.5.8 Rectangular ve Jagged Arrays

- Rectangular array, tüm satırların aynı sayıda öğeye ve tüm sütunların aynı sayıda öğeye sahip olduğu çok boyutlu bir array'dir.
- Jagged matris, değişen sayıda eleman içeren satırlara sahiptir.
  - Çok boyutlu array'ler aslında dizi dizileri (arrays of arrays) olarak göründüğünde mümkündür.
- C, C++ ve Java, jagged array'leri destekler.
- F# ve C# rectangular array'leri and jagged array'leri destekler.

Copyright © 2015 Pearson. All rights reserved.

6.5.9 Slices

- Bir slice, bir array'in bir alt yapısıdır; bir referans mekanizmasından başka bir şey değildir.
- Slices, array işlemleri olan dillerde kullanışlıdır.

Copyright © 2015 Pearson. All rights reserved.

6.5.9.1 Slice Örnekleri

- Python

```
vector = [2, 4, 6, 8, 10, 12, 14, 16]
mat = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

`vector (3:6)` is a üç elemanlı array

`mat[0][0:2]` `mat'`ın ilk satırının birinci ve ikinci elemanıdır

- Ruby, `slice` metodu ile slice'ı destekler

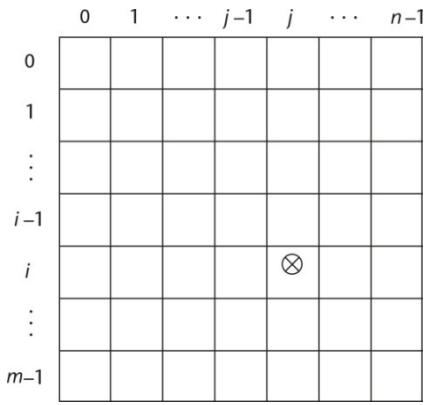
`list.slice(2, 2)` `list`'in üçüncü ve dördüncü öğelerini döndürür

Copyright © 2015 Pearson. All rights reserved.

6.5.10 Implementation of Arrays

- Erişim fonksiyonu, subscript ifadeleri dizideki bir adrese eşler.
- Tek boyutlu diziler için erişim fonksiyonu:

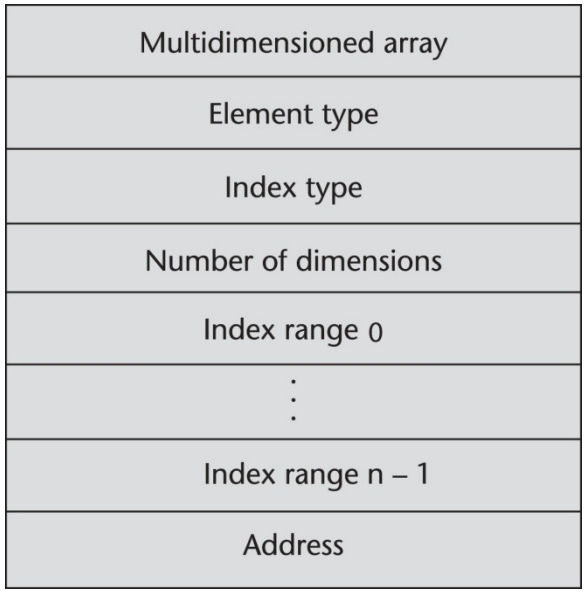
  address(list[k]) = address (list[lower_bound])
  + ((k-lower_bound) * element_size)



Copyright © 2015 Pearson. All rights reserved.

6.5.11 Accessing Multi-dimensioned Arrays

- İki yaygın yöntem:
  - Satır öncelikli sıralama (satırlara göre) – çoğu dilde kullanılır
  - Sütun öncelikli sıralama (sütunlara göre) – Fortran'da kullanılır
  - Çok boyutlu dizi için derleme zamanı tanımlayıcı

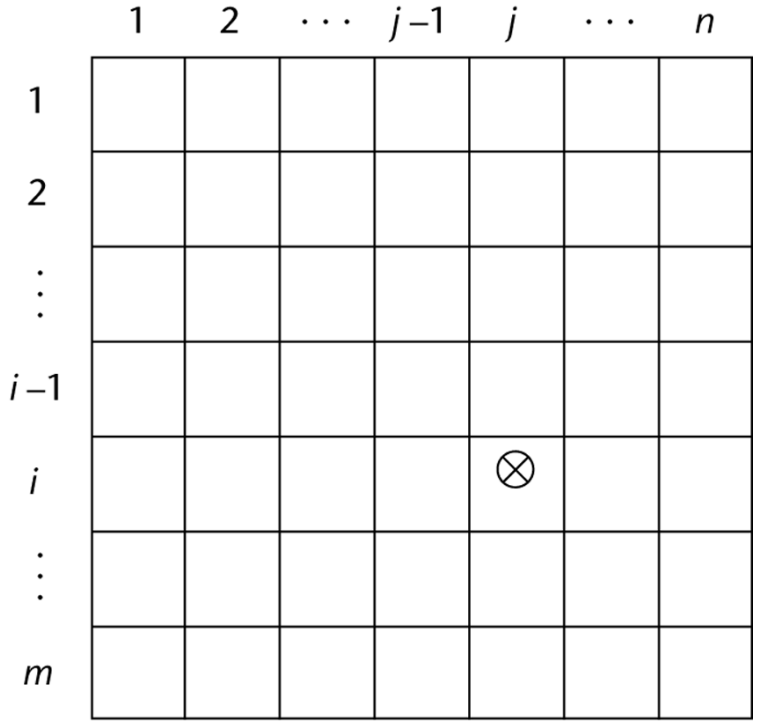


Copyright © 2015 Pearson. All rights reserved.

6.5.12 Çok Boyutlu Array'de Bir Eleman Bulma

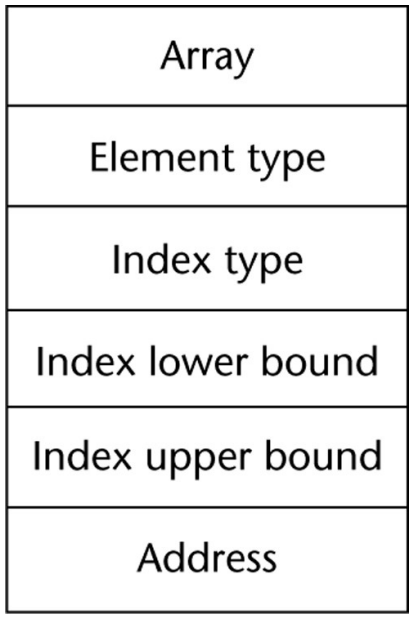
- Genel format

  Location (a[I,j]) = address of a [row_lb,col_lb] + (((I - row_lb) * n) + (j - col_lb)) * element_size

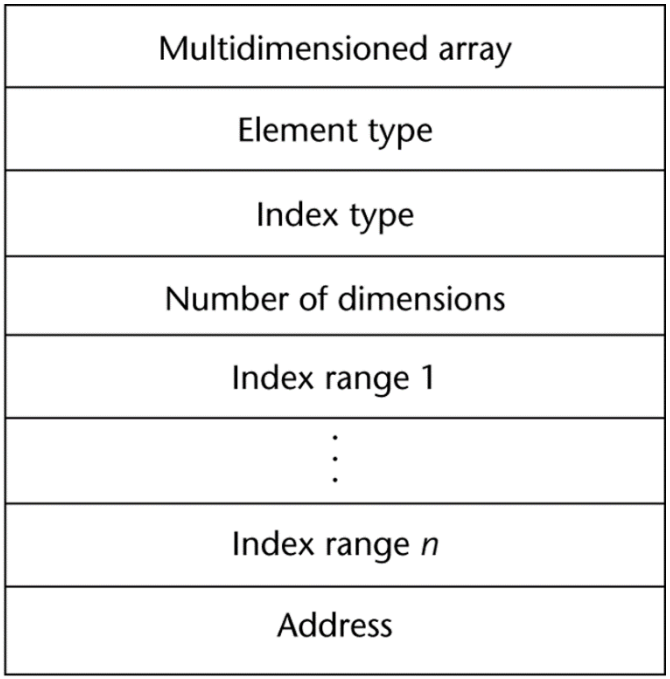


Copyright © 2015 Pearson. All rights reserved.

6.5.13 Compile-Time Descriptors



Tek Boyutlu Array



Çok Boyutlu Array

Copyright © 2015 Pearson. All rights reserved.

6.6 İlişkisel (Associative) Arrays

- Bir *ilişkisel (associative) array*, key adı verilen eşit sayıda değerle indekslenen, sıralanmamış veri öğelerinin bir koleksiyonudur.
  - Kullanıcı tanımlı keys saklanmalıdır
- Tasarım sorunları:

  - Elemanlara yapılan referansların biçimi nedir?

  - Boyut statik mi yoksa dinamik mi?
- Perl, Python, Ruby ve Lua'da yerleşik tür
  - Lua'da, bunlar tablolar tarafından desteklenir.

Copyright © 2015 Pearson. All rights reserved.

6.6.1 Associative Arrays in Perl

- Names % ile başlar; sabitler parantezlerle sınırlandırılır.

```
%hi_temps = ("Mon" => 77, "Tue" => 79, "Wed" =>
  65, …);
```

- Subscripting, köşeli parantezler ve keys kullanılarak yapılır.

```
$hi_temps{"Wed"} = 83;
```

  - Elemanlar **`delete`** ile kaldırılabilir

```
delete $hi_temps{"Tue"};
```

Copyright © 2015 Pearson. All rights reserved.

Record Types

- A *record* is a possibly heterogeneous aggregate of data elements in which the individual elements are identified by names
- Design issues:
  - What is the syntactic form of references to the field?
  - Are elliptical references allowed

Copyright © 2015 Pearson. All rights reserved.

Definition of Records in COBOL

- COBOL uses level numbers to show nested records; others use recursive definition

```
01 EMP-REC.
   02 EMP-NAME.
      05 FIRST PIC X(20).
      05 MID   PIC X(10).
      05 LAST  PIC X(20).
   02 HOURLY-RATE PIC 99V99.
```

Copyright © 2015 Pearson. All rights reserved.

References to Records

- Record field references
  1. COBOL
  field_name `OF` record_name_1 `OF` … `OF` record_name_n
  2. Others (dot notation)
  record_name_1.record_name_2. … record_name_n.field_name

- Fully qualified references must include all record names

- Elliptical references allow leaving out record names as long as the reference is unambiguous, for example in COBOL
  `FIRST, FIRST OF EMP-NAME`, and `FIRST` of EMP-REC are elliptical references to the employee's first name

Copyright © 2015 Pearson. All rights reserved.

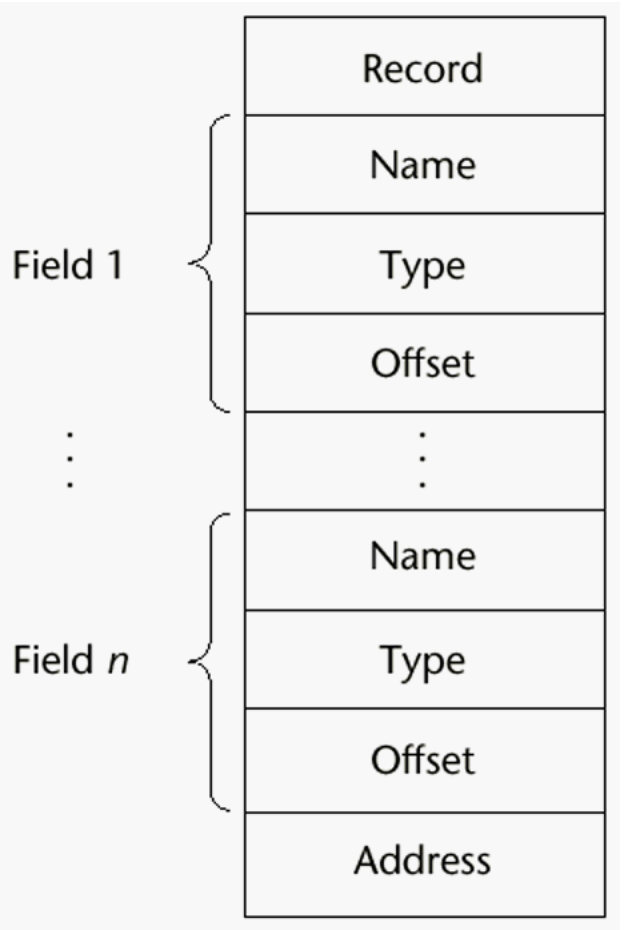
Evaluation and Comparison to Arrays

- Records are used when collection of data values is heterogeneous
- Access to array elements is much slower than access to record fields, because subscripts are dynamic (field names are static)
- Dynamic subscripts could be used with record field access, but it would disallow type checking and it would be much slower

Copyright © 2015 Pearson. All rights reserved.

Implementation of Record Type

Offset address relative to the beginning of the records is associated with each field



Copyright © 2015 Pearson. All rights reserved.

Tuple Types

- A tuple is a data type that is similar to a record, except that the elements are not named
- Used in Python, ML, and F# to allow functions to return multiple values
  - Python
    - Closely related to its lists, but immutable
    - Create with a tuple literal

      `myTuple = (3, 5.8, 'apple')`

      Referenced with subscripts (begin at 1)

    Catenation with + and deleted with **`del`**

Copyright © 2015 Pearson. All rights reserved.

Tuple Types (continued)

- ML

```
val myTuple = (3, 5.8, 'apple');
```

  - Access as follows:

  `#1(myTuple)` is the first element

  - A new tuple type can be defined

```
type intReal = int * real;
```

- F#

```
let tup = (3, 5, 7)
```

```
let a, b, c = tup
```

This assigns a tuple to a tuple pattern `(a, b, c)`

Copyright © 2015 Pearson. All rights reserved.

List Types

- Lists in Lisp and Scheme are delimited by parentheses and use no commas

  `(A B C D)` and `(A (B C) D)`

- Data and code have the same form

  As data, `(A B C)` is literally what it is

  As code, `(A B C)` is the function `A` applied to the parameters `B` and `C`

- The interpreter needs to know which a list is, so if it is data, we quote it with an apostrophe

  `'(A B C)` is data

Copyright © 2015 Pearson. All rights reserved.

List Types (continued)

- List Operations in Scheme
  - `CAR` returns the first element of its list parameter

    `(CAR '(A B C))` returns `A`
  - `CDR` returns the remainder of its list parameter after the first element has been removed

    `(CDR '(A B C))` returns `(B C)`
  - `CONS` puts its first parameter into its second parameter, a list, to make a new list

    `(CONS 'A (B C))` returns `(A B C)`
  - `LIST` returns a new list of its parameters

    `(LIST 'A 'B '(C D))` returns `(A B (C D))`

Copyright © 2015 Pearson. All rights reserved.

List Types (continued)

- List Operations in ML
  - Lists are written in brackets and the elements are separated by commas
  - List elements must be of the same type
  - The Scheme `CONS` function is a binary operator in ML, `::`

    `3 :: [5, 7, 9]` evaluates to `[3, 5, 7, 9]`
  - The Scheme `CAR` and `CDR` functions are named `hd` and `tl`, respectively

Copyright © 2015 Pearson. All rights reserved.

List Types (continued)

- F# Lists
  - Like those of ML, except elements are separated by semicolons and `hd` and `tl` are methods of the `List` class
- Python Lists
  - The list data type also serves as Python's arrays
  - Unlike Scheme, Common Lisp, ML, and F#, Python's lists are mutable
  - Elements can be of any type
  - Create a list with an assignment

```
myList = [3, 5.8, "grape"]
```

Copyright © 2015 Pearson. All rights reserved.

List Types (continued)

- Python Lists (continued)
  - List elements are referenced with subscripting, with indices beginning at zero

    `x = myList[1]` Sets `x` to `5.8`
  - List elements can be deleted with `del`

    ```
    del myList[1]
    ```
  - List Comprehensions – derived from set notation

    ```
    [x * x for x in range(6) if x % 3 == 0]
    ```

    `range(12)` creates `[0, 1, 2, 3, 4, 5, 6]`

    Constructed list: `[0, 9, 36]`

Copyright © 2015 Pearson. All rights reserved.

List Types (continued)

- Haskell's List Comprehensions
  - The original

```
[n * n | n <- [1..10]]
```

- F#'s List Comprehensions

```
let myArray = [|for i in 1 .. 5 -> [i * i) |]
```

- Both C# and Java supports lists through their generic heap-dynamic collection classes, `List` and `ArrayList`, respectively

Copyright © 2015 Pearson. All rights reserved.

Unions Types

- A *union* is a type whose variables are allowed to store different type values at different times during execution
- Design issue
  - Should type checking be required?

Copyright © 2015 Pearson. All rights reserved.

Discriminated vs. Free Unions

- C and C++ provide union constructs in which there is no language support for type checking; the union in these languages is called *free union*
- Type checking of unions require that each union include a type indicator called a *discriminant*
  - Supported by ML, Haskell, and F#

Copyright © 2015 Pearson. All rights reserved.

Unions in F#

- Defined with a type statement using OR

```
type intReal =
    | IntValue of int
    | RealValue of float;;
```

`intReal` is the new type

`IntValue` and `RealValue` are constructors

To create a value of type `intReal`:

```
let ir1 = IntValue 17;;
let ir2 = RealValue 3.4;;
```

Copyright © 2015 Pearson. All rights reserved.

Unions in F# (continued)

- Accessing the value of a union is done with pattern matching

```
match pattern with
    | expression_list1 -> expression1
    | ...
    | expression_listn -> expressionn
```

- Pattern can be any data type
- The expression list can have wild cards (_)

Copyright © 2015 Pearson. All rights reserved.

Unions in F# (continued)

Example:

```
let a = 7;;
let b = "grape";;
let x = match (a, b) with
      | 4, "apple" -> apple
      | _, "grape" -> grape
      | _ -> fruit;;
```

Copyright © 2015 Pearson. All rights reserved.

Unions in F# (continued)

To display the type of the `intReal` union:

```
let printType value =
  match value with
      | IntVale value -> printfn "int"
      | RealValue value -> printfn "float";;
```

If `ir1` and `ir2` are defined as previously,

`printType ir1` returns `int`

`printType ir2` returns `float`

Copyright © 2015 Pearson. All rights reserved.

Evaluation of Unions

- Free unions are unsafe
  - Do not allow type checking
- Java and C# do not support unions
  - Reflective of growing concerns for safety in programming language

Copyright © 2015 Pearson. All rights reserved.

Pointer and Reference Types

- A *pointer* type variable has a range of values that consists of memory addresses and a special value, *nil*
- Provide the power of indirect addressing
- Provide a way to manage dynamic memory
- A pointer can be used to access a location in the area where storage is dynamically created (usually called a *heap*)

Copyright © 2015 Pearson. All rights reserved.

Design Issues of Pointers

- What are the scope of and lifetime of a pointer variable?
- What is the lifetime of a heap-dynamic variable?
- Are pointers restricted as to the type of value to which they can point?
- Are pointers used for dynamic storage management, indirect addressing, or both?
- Should the language support pointer types, reference types, or both?

Copyright © 2015 Pearson. All rights reserved.

Pointer Operations

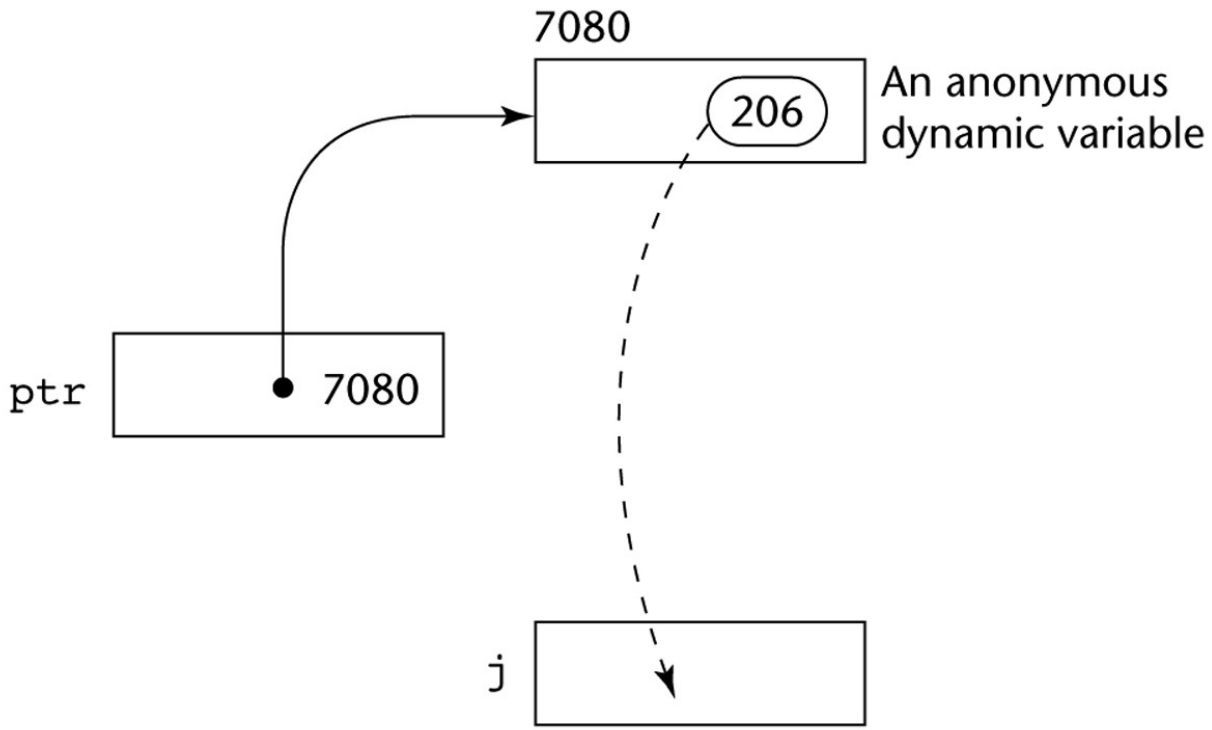
- Two fundamental operations: assignment and dereferencing
- Assignment is used to set a pointer variable's value to some useful address
- Dereferencing yields the value stored at the location represented by the pointer's value
  - Dereferencing can be explicit or implicit
  - C++ uses an explicit operation via *

    `j = *ptr`

    sets j to the value located at `ptr`

Copyright © 2015 Pearson. All rights reserved.

Pointer Assignment Illustrated



The assignment operation j = *ptr

Copyright © 2015 Pearson. All rights reserved.

Problems with Pointers

- Dangling pointers (dangerous)
  - A pointer points to a heap-dynamic variable that has been deallocated
- Lost heap-dynamic variable
  - An allocated heap-dynamic variable that is no longer accessible to the user program (often called *garbage*)
    - Pointer p1 is set to point to a newly created heap-dynamic variable
    - Pointer p1 is later set to point to another newly created heap-dynamic variable
    - The process of losing heap-dynamic variables is called *memory leakage*

Copyright © 2015 Pearson. All rights reserved.

Pointers in C and C++

- Extremely flexible but must be used with care
- Pointers can point at any variable regardless of when or where it was allocated
- Used for dynamic storage management and addressing
- Pointer arithmetic is possible
- Explicit dereferencing and address-of operators
- Domain type need not be fixed (**`void *`**)

  `void *` can point to any type and can be type checked (cannot be de-referenced)

Copyright © 2015 Pearson. All rights reserved.

Pointer Arithmetic in C and C++

```
float stuff[100];
float *p;
p = stuff;
```

`*(p+5)` is equivalent to `stuff[5]` and `p[5]`

`*(p+i)` is equivalent to `stuff[i]` and `p[i]`

Copyright © 2015 Pearson. All rights reserved.

Reference Types

- C++ includes a special kind of pointer type called a *reference type* that is used primarily for formal parameters
  - Advantages of both pass-by-reference and pass-by-value
- Java extends C++'s reference variables and allows them to replace pointers entirely
  - References are references to objects, rather than being addresses
- C# includes both the references of Java and the pointers of C++

Copyright © 2015 Pearson. All rights reserved.

Evaluation of Pointers

- Dangling pointers and dangling objects are problems as is heap management
- Pointers are like `goto`'s--they widen the range of cells that can be accessed by a variable
- Pointers or references are necessary for dynamic data structures--so we can't design a language without them

Copyright © 2015 Pearson. All rights reserved.

Representations of Pointers

- Large computers use single values
- Intel microprocessors use segment and offset

Copyright © 2015 Pearson. All rights reserved.

Dangling Pointer Problem

- *Tombstone*: extra heap cell that is a pointer to the heap-dynamic variable
  - The actual pointer variable points only at tombstones
  - When heap-dynamic variable de-allocated, tombstone remains but set to nil
  - Costly in time and space

. *Locks-and-keys*: Pointer values are represented as (key, address) pairs
  - Heap-dynamic variables are represented as variable plus cell for integer lock value
  - When heap-dynamic variable allocated, lock value is created and placed in lock cell and key cell of pointer

Copyright © 2015 Pearson. All rights reserved.

Heap Management

- A very complex run-time process
- Single-size cells vs. variable-size cells
- Two approaches to reclaim garbage
  - Reference counters (*eager approach*): reclamation is gradual
  - Mark-sweep (*lazy approach*): reclamation occurs when the list of variable space becomes empty

Copyright © 2015 Pearson. All rights reserved.

Reference Counter

- Reference counters: maintain a counter in every cell that store the number of pointers currently pointing at the cell
  - *Disadvantages*: space required, execution time required, complications for cells connected circularly
  - *Advantage*: it is intrinsically incremental, so significant delays in the application execution are avoided

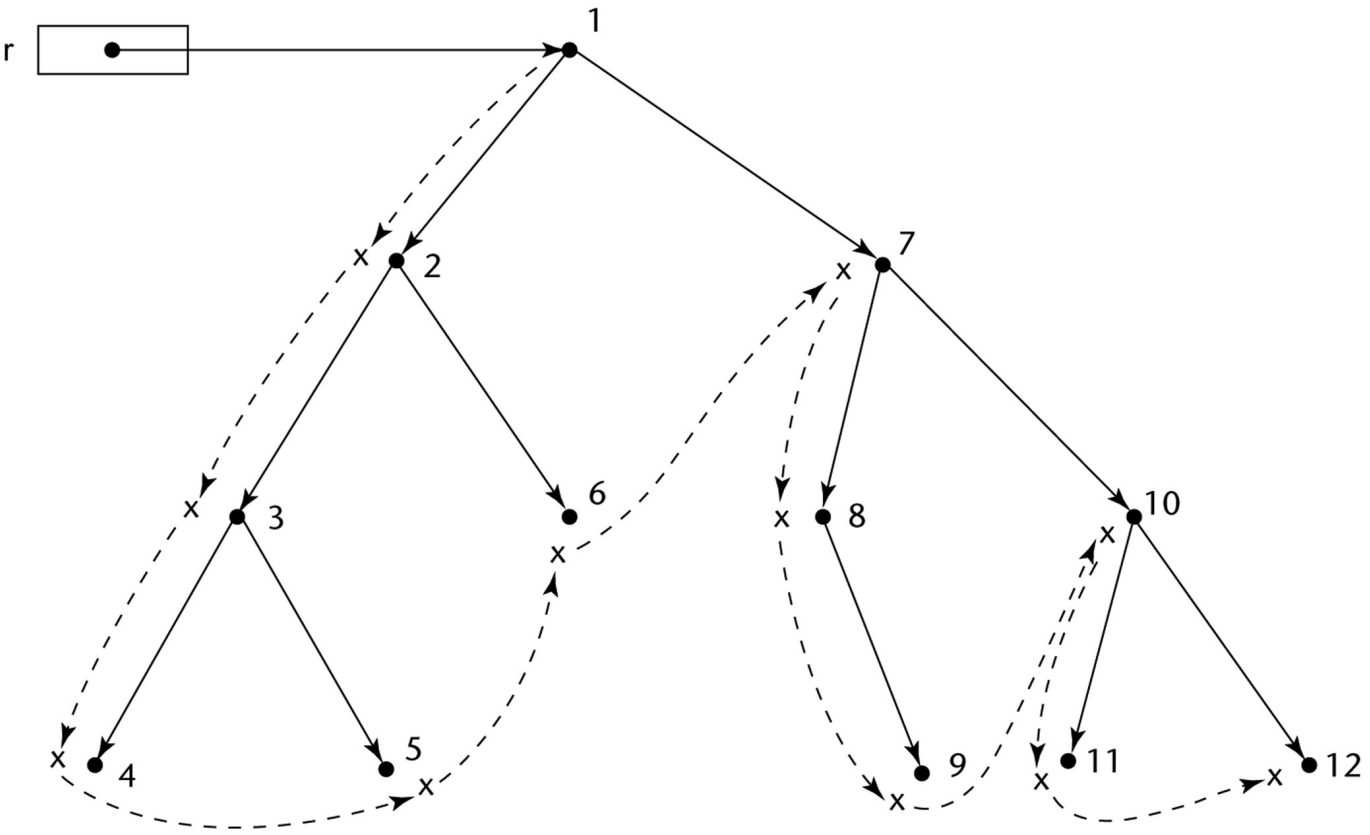
Copyright © 2015 Pearson. All rights reserved.

Mark-Sweep

- The run-time system allocates storage cells as requested and disconnects pointers from cells as necessary; mark-sweep then begins
  - Every heap cell has an extra bit used by collection algorithm
  - All cells initially set to garbage
  - All pointers traced into heap, and reachable cells marked as not garbage
  - All garbage cells returned to list of available cells
  - Disadvantages: in its original form, it was done too infrequently. When done, it caused significant delays in application execution. Contemporary mark-sweep algorithms avoid this by doing it more often—called incremental mark-sweep

Copyright © 2015 Pearson. All rights reserved.

Marking Algorithm



Dashed lines show the order of node_marking

Copyright © 2015 Pearson. All rights reserved.

Variable-Size Cells

- All the difficulties of single-size cells plus more
- Required by most programming languages
- If mark-sweep is used, additional problems occur
  - The initial setting of the indicators of all cells in the heap is difficult
  - The marking process in nontrivial
  - Maintaining the list of available space is another source of overhead

Copyright © 2015 Pearson. All rights reserved.

Type Checking

- Generalize the concept of operands and operators to include subprograms and assignments
- *Type checking* is the activity of ensuring that the operands of an operator are of compatible types
- A *compatible type* is one that is either legal for the operator, or is allowed under language rules to be implicitly converted, by compiler- generated code, to a legal type
  - This automatic conversion is called a *coercion*.
- A *type error* is the application of an operator to an operand of an inappropriate type

Copyright © 2015 Pearson. All rights reserved.

Type Checking (continued)

- If all type bindings are static, nearly all type checking can be static
- If type bindings are dynamic, type checking must be dynamic
- A programming language is *strongly typed* if type errors are always detected
- Advantage of strong typing: allows the detection of the misuses of variables that result in type errors

Copyright © 2015 Pearson. All rights reserved.

Strong Typing

Language examples:

- C and C++ are not: parameter type checking can be avoided; unions are not type checked
- Java and C# are, almost (because of explicit type casting)

- ML and F# are

Copyright © 2015 Pearson. All rights reserved.

Strong Typing (continued)

- Coercion rules strongly affect strong typing--they can weaken it considerably (C++ versus ML and F#)
- Although Java has just half the assignment coercions of C++, its strong typing is still far less effective than that of Ada

Copyright © 2015 Pearson. All rights reserved.

Name Type Equivalence

- *Name type equivalence* means the two variables have equivalent types if they are in either the same declaration or in declarations that use the same type name
- Easy to implement but highly restrictive:
  - Subranges of integer types are not equivalent with integer types
  - Formal parameters must be the same type as their corresponding actual parameters

Copyright © 2015 Pearson. All rights reserved.

Structure Type Equivalence

- *Structure type equivalence* means that two variables have equivalent types if their types have identical structures
- More flexible, but harder to implement

Copyright © 2015 Pearson. All rights reserved.

Type Equivalence (continued)

- Consider the problem of two structured types:
  - Are two record types equivalent if they are structurally the same but use different field names?
  - Are two array types equivalent if they are the same except that the subscripts are different?

    (e.g. `[1..10]` and `[0..9]`)
  - Are two enumeration types equivalent if their components are spelled differently?
  - With structural type equivalence, you cannot differentiate between types of the same structure
    (e.g. different units of speed, both float)

Copyright © 2015 Pearson. All rights reserved.

Theory and Data Types

- Type theory is a broad area of study in mathematics, logic, computer science, and philosophy
- Two branches of type theory in computer science:
  - Practical – data types in commercial languages
  - Abstract – typed lambda calculus

- A type system is a set of types and the rules that govern their use in programs

Copyright © 2015 Pearson. All rights reserved.

Theory and Data Types (continued)

- Formal model of a type system is a set of types and a collection of functions that define the type rules
  - Either an attribute grammar or a type map could be used for the functions
  - Finite mappings – model arrays and functions
  - Cartesian products – model tuples and records
  - Set unions – model union types
  - Subsets – model subtypes

Copyright © 2015 Pearson. All rights reserved.

Summary

- The data types of a language are a large part of what determines that language's style and usefulness
- The primitive data types of most imperative languages include numeric, character, and Boolean types
- The user-defined enumeration and subrange types are convenient and add to the readability and reliability of programs
- Arrays and records are included in most languages
- Pointers are used for addressing flexibility and to control dynamic storage management

Copyright © 2015 Pearson. All rights reserved.